API Docs

📋 Table of Contents

1. **Authentication**
2. **Delivery Order Status Flow**
3. **Complete API Reference**
    - **Step 1 — Get My Assigned Deliveries**
    - **Step 2 — Get Delivery Order Details**
    - **Step 3 — Confirm Full Delivery**
    - **Step 4 — Confirm Partial Delivery**
4. **Supporting APIs**
5. **Error Handling**
6. **Flutter / Dart Integration Guide**
7. **Full Mobile Screen Flow**

---

1. Authentication

All APIs require a **JWT Bearer Token**. The driver must first log in to receive a token.

🔐 Login API

```
POST /api/auth/login
Content-Type: application/json
```

Request Body:

```
json
{
"email":"driver@starenterprises.com",
"password":"yourpassword"
}
```

Response:

```
json
{
"token":"eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",
"role":"DRIVER",
"name":"Ahmed Khan",
```

```
"email":"driver@starenterprises.com"
}
```

> **Important:** Save the `token` in secure storage. Send it in every API call:
>
> ```
> Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...
> ```

---

2. Delivery Order Status Flow

```
CREATED → DISPATCHED → DELIVERED          (All items delivered)
                ↓
          PARTIALLY_DELIVERED             (Some items delivered)
                ↓
             CANCELLED
```

Status	Meaning	Who Sets It
`CREATED`	Order created, not yet dispatched	Admin / Manager
`DISPATCHED`	Order sent out for delivery	Admin / Manager
`IN_TRANSIT`	Order is on the way	System / Driver
`PARTIALLY_DELIVERED`	Only some garments were delivered ⚠️	**Driver (Partial Confirm)**
`DELIVERED`	All garments received by customer ✅	**Driver (Full Confirm)**
`CANCELLED`	Order was cancelled ❌	Admin / Manager

> ⚠️ **Note:** The driver can confirm delivery when status is `DISPATCHED` or `IN_TRANSIT`.
>
> **Full Delivery** → use `PUT /confirm` (no body needed)
>
> **Partial Delivery** → use `PUT /confirm-partial` with a JSON body listing which garment IDs were delivered

---

3. Complete API Reference

API 1: Get My Assigned Deliveries (Driver Login)

This is the **first screen** shown to the driver — the list of all their assigned deliveries.

```
GET /api/delivery-orders/my-deliveries
Authorization: Bearer {token}
```

No request body needed. The driver's identity is automatically resolved from the JWT token.

✅ Success Response — 200 OK:

```
json
[
  {
"id":1284,
"deliveryOrderNumber":"DO-2026-01284",
"orderHeaderId":751,
"customerId":12,
"customerName":"Mohammed Al-Rashidi",
"branchId":3,
"branchName":"Al Olaya Branch",
"driverId":45,
"driverName":"Ahmed Khan",
"status":"DISPATCHED",
"dispatchTime":"2026-05-25T08:30:00",
"deliveryTime":null,
"receivedBy":null,
"remarks":null,
"totalItems":5,
"createdAt":"2026-05-25T07:00:00",
"updatedAt":"2026-05-25T08:30:00",
"items": [
      {
"id":8209,
"garmentId":209,
"garmentQrCode":"QR-20260001",
"productId":3,
"productName":"Shirt",
"status":"OUT_FOR_DELIVERY"
      },
      {
"id":8210,
"garmentId":210,
"garmentQrCode":"QR-20260002",
"productId":3,
"productName":"Shirt",
"status":"OUT_FOR_DELIVERY"
      }
    ]
  }
]
```

❌ Error Responses:

Code	Reason
`401 Unauthorized`	Token missing or expired
`403 Forbidden`	Logged-in user is not a DRIVER

API 2: Get Delivery Order Details

Use this to load the **detail screen** for a single delivery order before confirming.

```
GET /api/delivery-orders/{id}
```

```
Authorization: Bearer {token}
```

Path Parameter:

Parameter	Type	Description
`id`	`Long`	The delivery order ID (e.g., `1284`)

Example:

```
GET /api/delivery-orders/1284
Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...
```

✅ Success Response — 200 OK:

```
json
{
"id":1284,
"deliveryOrderNumber":"DO-2026-01284",
"orderHeaderId":751,
"customerId":12,
"customerName":"Mohammed Al-Rashidi",
"branchId":3,
"branchName":"Al Olaya Branch",
"driverId":45,
"driverName":"Ahmed Khan",
"status":"DISPATCHED",
"dispatchTime":"2026-05-25T08:30:00",
"deliveryTime":null,
"receivedBy":null,
"remarks":null,
"totalItems":5,
"createdAt":"2026-05-25T07:00:00",
"updatedAt":"2026-05-25T08:30:00",
"items": [
    {
"id":8209,
"garmentId":209,
"garmentQrCode":"QR-20260001",
"productId":3,
"productName":"Shirt",
"status":"OUT_FOR_DELIVERY"
    },
    {
"id":8210,
"garmentId":210,
"garmentQrCode":"QR-20260002",
"productId":3,
"productName":"Shirt",
"status":"OUT_FOR_DELIVERY"
    },
    {
"id":8211,
"garmentId":211,
"garmentQrCode":"QR-20260003",
"productId":3,
"productName":"Shirt",
```

```
"status":"OUT_FOR_DELIVERY"
    },
    {
"id":8212,
"garmentId":212,
"garmentQrCode":"QR-20260004",
"productId":3,
"productName":"Shirt",
"status":"OUT_FOR_DELIVERY"
    },
    {
"id":8213,
"garmentId":213,
"garmentQrCode":"QR-20260005",
"productId":3,
"productName":"Shirt",
"status":"OUT_FOR_DELIVERY"
    }
  ]
}
```


❌ Error Responses:

Code	Reason
`401 Unauthorized`	Token missing or expired
`404 Not Found`	No delivery order with that ID

API 3: Confirm Full Delivery ⭐ (All Items)

Called when the driver delivers **all items** to the customer. Marks the entire order as `DELIVERED`.

```
PUT /api/delivery-orders/{id}/confirm
Authorization: Bearer {token}
```

Query Parameters (in URL):

Parameter	Type	Required	Description
`receivedBy`	`String`	❌ Optional	Name of person who received
`remarks`	`String`	❌ Optional	Delivery notes

Example:

```
PUT /api/delivery-orders/1284/confirm?receivedBy=Mohammed&remarks=Delivered+at+door
Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...
```

> No JSON body needed. All items in the order are confirmed.

✅ Success Response — 200 OK:

```
json
{
"id":1284,
"status":"DELIVERED",
"deliveryTime":"2026-05-25T10:15:00",
"receivedBy":"Mohammed Al-Rashidi",
"totalItems":5,
"items": [
    {"id":8209,"garmentId":209,"productName":"Shirt","status":"DELIVERED" },
    {"id":8210,"garmentId":210,"productName":"Shirt","status":"DELIVERED" },
    {"id":8211,"garmentId":211,"productName":"Shirt","status":"DELIVERED" },
    {"id":8212,"garmentId":212,"productName":"Shirt","status":"DELIVERED" },
    {"id":8213,"garmentId":213,"productName":"Shirt","status":"DELIVERED" }
  ]
}
```

What Happens:

1. ✅ Order status → `DELIVERED`
2. ✅ All garments → `DELIVERED`
3. ✅ `deliveryTime` set to now
4. ✅ Order Header synced → `DELIVERED`
5. ✅ Ready dashboard synced

❌ **Errors:**

Code	Reason
`401`	Token expired
`403`	Not DRIVER / ADMIN / MANAGER
`404`	Wrong delivery ID
`500`	Status not DISPATCHED or IN_TRANSIT

API 4: Confirm Partial Delivery 🔲 (Like Web Modal)

This is the **partial delivery API** — exactly matching what your web modal does. The driver selects **which garments** were handed to the customer using checkboxes. Unselected items remain `OUT_FOR_DELIVERY`.

> **Rule:** If 3 out of 5 items are selected → order becomes `PARTIALLY_DELIVERED`. If all 5 are selected → order becomes `DELIVERED`.

```
PUT /api/delivery-orders/{id}/confirm-partial
Authorization: Bearer {token}
Content-Type: application/json
```

Path Parameter:

Parameter	Type	Required	Description
`id`	`Long`	✅ Yes	Delivery Order ID

Query Parameters (optional, in URL):

Parameter	Type	Required	Description
`receivedBy`	`String`	❌ Optional	Name of customer who received
`remarks`	`String`	❌ Optional	Notes about the partial delivery

Request Body (JSON):

```
json
{
"garmentIds": [209,210,211],
"proofImageUrl":"https://your-storage.com/proof/1284.jpg"
}
```

Field	Type	Required	Description
`garmentIds`	`Set<Long>`	✅ Yes	IDs of **garments** that were delivered (from `items[].garmentId`)
`proofImageUrl`	`String`	❌ Optional	URL of the photo taken as proof of delivery

Full Example — 3 of 5 items delivered:

```
PUT /api/delivery-orders/1284/confirm-partial?receivedBy=Mohammed&remarks=2+items+not+available
Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...
Content-Type: application/json

{
  "garmentIds": [209, 210, 211],
  "proofImageUrl": "https://cdn.star.com/deliveries/1284_proof.jpg"
}
```

✅ Success Response — 200 OK (Partial):

Notice: garments 209, 210, 211 → `DELIVERED`. Garments 212, 213 remain `OUT_FOR_DELIVERY`.

```
json
{
"id":1284,
"deliveryOrderNumber":"DO-2026-01284",
"customerId":12,
"customerName":"Mohammed Al-Rashidi",
"driverName":"Ahmed Khan",
"status":"PARTIALLY_DELIVERED",
"dispatchTime":"2026-05-25T08:30:00",
"deliveryTime":"2026-05-25T10:15:00",
"receivedBy":"Mohammed Al-Rashidi",
"remarks":"Partial delivery note: 2 items not available",
"totalItems":5,
```

```
"items": [
    {"id":8209,"garmentId":209,"productName":"Shirt","status":"DELIVERED" },
    {"id":8210,"garmentId":210,"productName":"Shirt","status":"DELIVERED" },
    {"id":8211,"garmentId":211,"productName":"Shirt","status":"DELIVERED" },
    {"id":8212,"garmentId":212,"productName":"Shirt","status":"INCLUDED" },
    {"id":8213,"garmentId":213,"productName":"Shirt","status":"INCLUDED" }
  ]
}
```


✅ Success Response — 200 OK (All Selected = Full Delivery):

If all 5 garment IDs are passed, order becomes `DELIVERED` automatically:


```
json
{
"id":1284,
"status":"DELIVERED",
"items": [
    {"id":8209,"garmentId":209,"status":"DELIVERED" },
    {"id":8210,"garmentId":210,"status":"DELIVERED" },
    {"id":8211,"garmentId":211,"status":"DELIVERED" },
    {"id":8212,"garmentId":212,"status":"DELIVERED" },
    {"id":8213,"garmentId":213,"status":"DELIVERED" }
  ]
}
```


What Happens Internally:

1. ✅ Only garments in `garmentIds` → `DELIVERED`
2. ✅ Remaining garments stay as `OUT_FOR_DELIVERY`
3. ✅ If all garments delivered → Order status `DELIVERED`
4. ✅ If some remaining → Order status `PARTIALLY_DELIVERED`
5. ✅ Garment history logs created for each delivered item
6. ✅ Order Header status synced (`DELIVERED` or `PARTIALLY_DELIVERED`)
7. ✅ Ready dashboard synced
8. ✅ `proofImageUrl` appended to remarks

❌ Errors:

Code	Message	Reason
`401`	—	Token expired
`403`	—	Not DRIVER / ADMIN / MANAGER
`404`	`"Delivery Order not found: 1284"`	Wrong ID
`500`	`"must be DISPATCHED or IN_TRANSIT"`	Wrong status

Code	Message	Reason
`500`	`"At least one garment ID must be selected"`	Empty `garmentIds`

How to Decide Which API to Call

```
Driver opens the delivery modal
        ↓
Check all items? ——— YES ——→ PUT /confirm        (full delivery)
        ↓
       NO
        ↓
Select individual items → PUT /confirm-partial    (partial delivery)
```

4. Supporting APIs

Get Deliveries by Driver ID (Admin use)

If you need to load a specific driver's deliveries from the admin panel:

```
GET /api/delivery-orders/driver/{driverId}
Authorization: Bearer {token}
```

Example:

```
GET /api/delivery-orders/driver/45
Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...
```

Returns same list format as `my-deliveries`.

Get Deliveries by Status

Filter deliveries by status (for admin dashboards):

```
GET /api/delivery-orders/status/{status}
Authorization: Bearer {token}
```

Valid status values: `CREATED`, `DISPATCHED`, `IN_TRANSIT`, `PARTIALLY_DELIVERED`, `DELIVERED`, `CANCELLED`

Example:

```
GET /api/delivery-orders/status/DISPATCHED
```

```
Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...
```

Download Delivery PDF

Generate and download a PDF receipt for a delivery order:

```
GET /api/delivery-orders/{id}/pdf
Authorization: Bearer {token}
```

Example:

```
GET /api/delivery-orders/1284/pdf
Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...
```

Response: Returns a PDF file (Content-Type: `application/pdf`).

5. Error Handling

Common Error Scenarios for Mobile

Scenario	API Response	App Action
Token expired	`401 Unauthorized`	Redirect to Login screen
Wrong delivery ID	`404 + "Delivery Order not found"`	Show error toast
Delivery not dispatched	`500 + "must be dispatched"`	Show "Cannot confirm yet" message
Network timeout	No response	Show "No internet" message
Already confirmed	Status is `DELIVERED`	Show "Already delivered"

Error Response Format

```
json
{
"timestamp":"2026-05-25T10:00:00",
"status":500,
"error":"Internal Server Error",
"message":"Delivery order must be dispatched before it can be confirmed",
"path":"/api/delivery-orders/1284/confirm"
}
```

6. React Native Integration Guide

> Uses **axios** for HTTP requests and **AsyncStorage** for token storage. Install dependencies: `npm install axios @react-native-async-storage/async-storage`

Step 1 — Setup: Token Storage Helper

```
ts
// src/utils/storage.ts
importAsyncStoragefrom'@react-native-async-storage/async-storage';

exportconstsaveToken=async (token:string)=> {
awaitAsyncStorage.setItem('auth_token',token);
};

exportconstgetToken=async ():Promise<string|null>=> {
returnAsyncStorage.getItem('auth_token');
};

exportconstremoveToken=async ()=> {
awaitAsyncStorage.removeItem('auth_token');
};
```

Step 2 — Axios Instance with Auth Header

```
ts
// src/api/axiosInstance.ts
importaxiosfrom'axios';
importAsyncStoragefrom'@react-native-async-storage/async-storage';

// 🔧 Change this to your server IP / production URL
constBASE_URL='http://192.168.1.100:8080';

constapi=axios.create({baseURL:BASE_URL });

// Auto-attach token to every request
api.interceptors.request.use(async (config)=> {
consttoken=awaitAsyncStorage.getItem('auth_token');
if (token) {
config.headers.Authorization=`Bearer${token}`;
  }
returnconfig;
});

// Handle 401 globally — redirect to login
api.interceptors.response.use(
  (response)=>response,
  (error)=> {
if (error.response?.status===401) {
// Navigate to Login screen — implement your own navigation logic
console.warn('Session expired — redirect to login');
    }
returnPromise.reject(error);
  }
);
```

```
exportdefaultapi;
```

Step 3 — TypeScript Types

```
ts
// src/types/delivery.ts

exportinterfaceDeliveryItem {
id:number;
garmentId:number;
garmentQrCode:string;
productId:number;
productName:string;
status:'OUT_FOR_DELIVERY'|'DELIVERED'|'INCLUDED';
}

exportinterfaceDeliveryOrder {
id:number;
deliveryOrderNumber:string;
orderHeaderId:number;
customerId:number;
customerName:string;
branchId?:number;
branchName?:string;
driverId:number;
driverName:string;
status:'CREATED'|'DISPATCHED'|'IN_TRANSIT'|'PARTIALLY_DELIVERED'|'DELIVERED'|'CANCELLED';
dispatchTime?:string;
deliveryTime?:string;
receivedBy?:string;
remarks?:string;
totalItems:number;
items:DeliveryItem[];
createdAt:string;
updatedAt:string;
}

exportinterfacePartialDeliveryRequest {
garmentIds:number[];// garmentId of each selected item
proofImageUrl?:string;// optional photo proof URL
}
```

Step 4 — Delivery API Service

```
ts
// src/api/deliveryService.ts
importapifrom'./axiosInstance';
import {DeliveryOrder,PartialDeliveryRequest }from'../types/delivery';

// ✅ Login
exportconstlogin=async (email:string,password:string)=> {
constres=awaitapi.post('/api/auth/login', {email,password });
returnres.data;// { token, role, name, email }
};
```

```
// ✅ Get all deliveries for the logged-in driver
exportconstgetMyDeliveries=async ():Promise<DeliveryOrder[]>=> {
constres=awaitapi.get('/api/delivery-orders/my-deliveries');
returnres.data;
};

// ✅ Get single delivery order detail
exportconstgetDeliveryById=async (id:number):Promise<DeliveryOrder>=> {
constres=awaitapi.get(`/api/delivery-orders/${id}`);
returnres.data;
};

// ✅ Confirm FULL delivery — all items delivered
exportconstconfirmDelivery=async (
id:number,
receivedBy?:string,
remarks?:string,
):Promise<DeliveryOrder>=> {
constparams:Record<string,string>= {};
if (receivedBy)params.receivedBy=receivedBy;
if (remarks)params.remarks=remarks;

constres=awaitapi.put(`/api/delivery-orders/${id}/confirm`,null, {params });
returnres.data;
};

// ✅ Confirm PARTIAL delivery — selected garments only (matches web modal checkboxes)
exportconstconfirmPartialDelivery=async (
id:number,
body:PartialDeliveryRequest,
receivedBy?:string,
remarks?:string,
):Promise<DeliveryOrder>=> {
constparams:Record<string,string>= {};
if (receivedBy)params.receivedBy=receivedBy;
if (remarks)params.remarks=remarks;

constres=awaitapi.put(`/api/delivery-orders/${id}/confirm-partial`,body, {params });
returnres.data;
};
```

Step 5 — Complete Confirm Delivery Screen (with Partial Support)

This screen mirrors your web modal: shows all items as checkboxes, the driver selects which ones were delivered, then taps **Confirm Delivery**.

```
tsx
// src/screens/ConfirmDeliveryScreen.tsx
importReact, {useState }from'react';
import {
View,Text,ScrollView,TextInput,TouchableOpacity,
StyleSheet,Alert,ActivityIndicator,FlatList
}from'react-native';
import {confirmDelivery,confirmPartialDelivery }from'../api/deliveryService';
import {DeliveryOrder,DeliveryItem }from'../types/delivery';

interfaceProps {
```

```
route: {params: {delivery:DeliveryOrder } };
navigation:any;
}

exportdefaultfunctionConfirmDeliveryScreen({route,navigation }:Props) {
const {delivery }=route.params;

// Track which garments are checked (default: all checked)
const [checkedIds,setCheckedIds]=useState<Set<number>>(
newSet(delivery.items.map((item)=>item.garmentId))
  );
const [receivedBy,setReceivedBy]=useState('');
const [remarks,setRemarks]=useState('');
const [loading,setLoading]=useState(false);

constisAllSelected=checkedIds.size===delivery.items.length;
constselectedCount=checkedIds.size;

consttoggleItem= (garmentId:number)=> {
setCheckedIds((prev)=> {
constnext=newSet(prev);
next.has(garmentId)?next.delete(garmentId):next.add(garmentId);
returnnext;
    });
  };

consttoggleAll= ()=> {
if (isAllSelected) {
setCheckedIds(newSet());// deselect all
    }else {
setCheckedIds(newSet(delivery.items.map((i)=>i.garmentId)));// select all
    }
  };

consthandleConfirm=async ()=> {
if (checkedIds.size===0) {
Alert.alert('Error','Please select at least one item to deliver.');
return;
    }

setLoading(true);
try {
letresult:DeliveryOrder;

if (isAllSelected) {
// All items selected → use full confirm endpoint
result=awaitconfirmDelivery(delivery.id,receivedBy,remarks);
      }else {
// Some items selected → use partial confirm endpoint
result=awaitconfirmPartialDelivery(
delivery.id,
          {garmentIds:Array.from(checkedIds) },
receivedBy,
remarks,
        );
      }

constisPartial=result.status==='PARTIALLY_DELIVERED';

Alert.alert(
isPartial?'⚠ Partially Delivered':'✅ Delivery Confirmed',
isPartial
```

```
?`${selectedCount} of${delivery.totalItems} items delivered. Order marked as Partially Delivered.
`
:'All items have been delivered successfully!',
        [{text:'OK',onPress: ()=>navigation.goBack() }],
      );
    }catch (error:any) {
constmsg=error?.response?.data?.message||'Failed to confirm delivery.';
Alert.alert('Error',msg);
    }finally {
setLoading(false);
    }
  };

constrenderItem= ({item }: {item:DeliveryItem })=> {
constisChecked=checkedIds.has(item.garmentId);
return (
<TouchableOpacity
style={[styles.itemRow,isChecked &&styles.itemRowChecked]}
onPress={()=>toggleItem(item.garmentId)}
activeOpacity={0.7}
>
{/* Checkbox */}
<Viewstyle={[styles.checkbox,isChecked &&styles.checkboxChecked]}>
{isChecked &&<Textstyle={styles.checkmark}>✓</Text>}
</View>

{/* Item Info */}
<Viewstyle={styles.itemInfo}>
<Textstyle={styles.itemId}>#{item.id}</Text>
<Textstyle={styles.itemProduct}>{item.productName}</Text>
<Textstyle={styles.itemQr}>{item.garmentQrCode}</Text>
</View>

{/* Status Badge */}
<Viewstyle={styles.statusBadge}>
<Textstyle={styles.statusText}>{item.status}</Text>
</View>
</TouchableOpacity>
    );
  };

constisDeliverable=
delivery.status==='DISPATCHED'||
delivery.status==='IN_TRANSIT'||
delivery.status==='PARTIALLY_DELIVERED';

return (
<Viewstyle={styles.container}>
{/* Header */}
<Viewstyle={styles.header}>
<Textstyle={styles.headerTitle}>Complete Delivery</Text>
<Textstyle={styles.headerSubtitle}>
          Order #{delivery.id} —{delivery.customerName}
</Text>
</View>

<ScrollViewcontentContainerStyle={styles.body}>
{/* Info Banner */}
<Viewstyle={styles.infoBanner}>
<Textstyle={styles.infoText}>
            Select the garments that have been delivered to the customer.
            If only some items are selected, the order will be marked as{' '}
```

```
<Textstyle={styles.bold}>Partially Delivered</Text>.
</Text>
</View>

{/* Select All Row */}
<TouchableOpacitystyle={styles.selectAllRow}onPress={toggleAll}>
<Viewstyle={[styles.checkbox,isAllSelected &&styles.checkboxChecked]}>
{isAllSelected &&<Textstyle={styles.checkmark}>✓</Text>}
</View>
<Textstyle={styles.selectAllText}>
{isAllSelected ?'Deselect All' :'Select All'}
</Text>
<Textstyle={styles.countBadge}>
{selectedCount} /{delivery.totalItems} selected
</Text>
</TouchableOpacity>

{/* Garment List */}
<FlatList
data={delivery.items}
keyExtractor={(item)=>item.id.toString()}
renderItem={renderItem}
scrollEnabled={false}
/>

{/* Received By */}
<Textstyle={styles.label}>Received By</Text>
<TextInput
style={styles.input}
placeholder="Customer name who received items"
value={receivedBy}
onChangeText={setReceivedBy}
/>

{/* Remarks */}
<Textstyle={styles.label}>Remarks (Optional)</Text>
<TextInput
style={[styles.input,styles.textArea]}
placeholder="Any notes about the delivery..."
value={remarks}
onChangeText={setRemarks}
multiline
numberOfLines={3}
/>

{/* Status Preview */}
{selectedCount>0 && (
<Viewstyle={[
styles.previewBanner,
isAllSelected ?styles.previewGreen :styles.previewOrange,
          ]}>
<Textstyle={styles.previewText}>
{isAllSelected
                ?`✅${selectedCount} items selected → Order will be DELIVERED`
                :`⚠${selectedCount} of${delivery.totalItems} items → Order will be PARTIALLY DELI
VERED`}
</Text>
</View>
        )}

{/* Confirm Button */}
{isDeliverable ? (
```

```
<TouchableOpacity
style={[styles.confirmBtn, (loading ||checkedIds.size ===0) &&styles.btnDisabled]}
onPress={handleConfirm}
disabled={loading ||checkedIds.size ===0}
>
{loading ? (
<ActivityIndicatorcolor="#fff"/>
            ) : (
<Textstyle={styles.confirmBtnText}>Confirm Delivery</Text>
            )}
</TouchableOpacity>
        ) : (
<Viewstyle={styles.alreadyDone}>
<Textstyle={styles.alreadyDoneText}>
{delivery.status ==='DELIVERED'
                ?'✅ This delivery is already completed.'
                :'❌ This delivery cannot be confirmed.'}
</Text>
</View>
        )}
</ScrollView>
</View>
  );
}

conststyles=StyleSheet.create({
container: {flex:1,backgroundColor:'#F5F7FA' },
header: {backgroundColor:'#1A237E',padding:20,paddingTop:50 },
headerTitle: {color:'#fff',fontSize:20,fontWeight:'bold' },
headerSubtitle: {color:'#C5CAE9',fontSize:14,marginTop:4 },
body: {padding:16,paddingBottom:40 },
infoBanner: {
backgroundColor:'#E8EAF6',borderRadius:8,
padding:12,marginBottom:16,
  },
infoText: {color:'#3949AB',fontSize:13,lineHeight:20 },
bold: {fontWeight:'bold' },
selectAllRow: {
flexDirection:'row',alignItems:'center',
backgroundColor:'#fff',borderRadius:8,
padding:12,marginBottom:8,
borderWidth:1,borderColor:'#C5CAE9',
  },
selectAllText: {flex:1,marginLeft:12,fontWeight:'600',color:'#1A237E' },
countBadge: {color:'#7986CB',fontSize:13 },
itemRow: {
flexDirection:'row',alignItems:'center',
backgroundColor:'#fff',borderRadius:8,
padding:12,marginBottom:6,
borderWidth:1,borderColor:'#E0E0E0',
  },
itemRowChecked: {borderColor:'#3F51B5',backgroundColor:'#F3F4FF' },
checkbox: {
width:22,height:22,borderRadius:4,
borderWidth:2,borderColor:'#9E9E9E',
alignItems:'center',justifyContent:'center',
  },
checkboxChecked: {backgroundColor:'#3F51B5',borderColor:'#3F51B5' },
checkmark: {color:'#fff',fontSize:14,fontWeight:'bold' },
itemInfo: {flex:1,marginLeft:12 },
itemId: {fontSize:12,color:'#9E9E9E' },
itemProduct: {fontSize:15,fontWeight:'600',color:'#212121' },
```

```
itemQr: {fontSize:11,color:'#BDBDBD',marginTop:2 },
statusBadge: {
backgroundColor:'#1A237E',borderRadius:6,
paddingHorizontal:8,paddingVertical:4,
  },
statusText: {color:'#fff',fontSize:10,fontWeight:'600' },
label: {fontSize:14,fontWeight:'600',color:'#424242',marginTop:16,marginBottom:6 },
input: {
backgroundColor:'#fff',borderRadius:8,
borderWidth:1,borderColor:'#E0E0E0',
padding:12,fontSize:14,color:'#212121',
  },
textArea: {height:80,textAlignVertical:'top' },
previewBanner: {borderRadius:8,padding:12,marginTop:16 },
previewGreen: {backgroundColor:'#E8F5E9' },
previewOrange: {backgroundColor:'#FFF8E1' },
previewText: {fontSize:13,fontWeight:'600' },
confirmBtn: {
backgroundColor:'#1A237E',borderRadius:10,
padding:16,alignItems:'center',marginTop:24,
  },
btnDisabled: {backgroundColor:'#9E9E9E' },
confirmBtnText: {color:'#fff',fontSize:16,fontWeight:'bold' },
alreadyDone: {
backgroundColor:'#F5F5F5',borderRadius:8,
padding:16,alignItems:'center',marginTop:24,
  },
alreadyDoneText: {color:'#757575',fontSize:14 },
});
```

Step 6 — Navigation Setup (React Navigation)

```
tsx
// src/navigation/DeliveryStack.tsx
importReactfrom'react';
import {createNativeStackNavigator }from'@react-navigation/native-stack';
importMyDeliveriesScreenfrom'../screens/MyDeliveriesScreen';
importConfirmDeliveryScreenfrom'../screens/ConfirmDeliveryScreen';

constStack=createNativeStackNavigator();

exportdefaultfunctionDeliveryStack() {
return (
<Stack.NavigatorscreenOptions={{headerShown:false }}>
<Stack.Screenname="MyDeliveries"component={MyDeliveriesScreen}/>
<Stack.Screenname="ConfirmDelivery"component={ConfirmDeliveryScreen}/>
</Stack.Navigator>
  );
}
```

Navigate to the screen like this from your list:

```
tsx
navigation.navigate('ConfirmDelivery', {delivery:selectedDeliveryOrder });
```

7. Full Mobile Screen Flow

```
📱 DRIVER APP FLOW (React Native)
═══════════════════════════════════════════════

1. Login Screen
   → POST /api/auth/login
   → Save JWT token in AsyncStorage

2. My Deliveries List Screen
   → GET /api/delivery-orders/my-deliveries
   → Show list of assigned delivery orders

3. Confirm Delivery Screen (tap an order)
   → GET /api/delivery-orders/{id}  (load fresh details)
   → Show garment list with checkboxes (all checked by default)
   → Driver unchecks items NOT given to customer
   → Enter "Received By" name + optional remarks

4. Tap "Confirm Delivery"
   → ALL checked?  → PUT /confirm        (status = DELIVERED)
   → SOME checked? → PUT /confirm-partial (status = PARTIALLY_DELIVERED)

5. Success Alert → Go back to My Deliveries list
```

---

8. Quick Reference — cURL Examples

Login

```
bash
curl-XPOSThttp://localhost:8080/api/auth/login\
-H"Content-Type: application/json"\
-d'{"email":"driver@star.com","password":"password123"}'
```

Get My Deliveries

```
bash
curl-XGEThttp://localhost:8080/api/delivery-orders/my-deliveries\
-H"Authorization: Bearer YOUR_TOKEN"
```

Confirm Full Delivery

```
bash
curl-XPUT"http://localhost:8080/api/delivery-orders/1284/confirm?receivedBy=Mohammed&remarks=Deli
vered+at+door"\
-H"Authorization: Bearer YOUR_TOKEN"
```

Confirm Partial Delivery (3 of 5 items)

```
bash
curl-XPUT"http://localhost:8080/api/delivery-orders/1284/confirm-partial?receivedBy=Mohammed&rema
rks=2+items+missing"\
-H"Authorization: Bearer YOUR_TOKEN"\
-H"Content-Type: application/json"\
-d'{"garmentIds":[209,210,211],"proofImageUrl":"https://cdn.star.com/proof.jpg"}'
```

Download PDF Receipt

```
bash
curl-XGEThttp://localhost:8080/api/delivery-orders/1284/pdf\
-H"Authorization: Bearer YOUR_TOKEN"\
-odelivery_1284.pdf
```

---

9. Summary Table — All Delivery APIs

#	Method	Endpoint	Auth	Purpose
1	`POST`	`/api/auth/login`	None	Driver login, get JWT token
2	`GET`	`/api/delivery-orders/my-deliveries`	DRIVER	Get all deliveries assigned to me
3	`GET`	`/api/delivery-orders/{id}`	Any	Get single delivery order detail
4	`PUT`	`/api/delivery-orders/{id}/confirm`	DRIVER / ADMIN	⭐ Confirm ALL items delivered
5	`PUT`	`/api/delivery-orders/{id}/confirm-partial`	DRIVER / ADMIN	🔲 Confirm SELECTED items (partial)
6	`GET`	`/api/delivery-orders/driver/{driverId}`	Any	Get deliveries by driver ID
7	`GET`	`/api/delivery-orders/status/{status}`	Any	Filter by status
8	`GET`	`/api/delivery-orders/{id}/pdf`	Any	Download PDF receipt

Generated for Star Enterprises Laundry Management System — v1.0 React Native | Spring Boot API

Step 1 — Create API Service

```
dart
// lib/services/delivery_api_service.dart

import'dart:convert';
import'package:http/http.dart'as http;

classDeliveryApiService {
// 🔧 Change this to your server IP
staticconstString baseUrl='http://192.168.1.100:8080';

finalString token;

DeliveryApiService({requiredthis.token});

Map<String,String>get _headers=> {
'Authorization':'Bearer$token',
'Content-Type':'application/json',
  };

// ✅ Get my deliveries (driver only)
Future<List<dynamic>>getMyDeliveries()async {
final response=await http.get(
Uri.parse('$baseUrl/api/delivery-orders/my-deliveries'),
      headers: _headers,
    );

if (response.statusCode==200) {
returnjsonDecode(response.body)asList<dynamic>;
    }elseif (response.statusCode==401) {
throwException('Session expired. Please login again.');
    }else {
throwException('Failed to load deliveries:${response.statusCode}');
    }
  }

// ✅ Get single delivery order details
Future<Map<String,dynamic>>getDeliveryById(int id)async {
final response=await http.get(
Uri.parse('$baseUrl/api/delivery-orders/$id'),
      headers: _headers,
    );

if (response.statusCode==200) {
returnjsonDecode(response.body)asMap<String,dynamic>;
    }else {
throwException('Delivery not found');
    }
  }

// ✅ Confirm Full Delivery — all items
Future<Map<String,dynamic>>confirmDelivery({
requiredint deliveryOrderId,
String? receivedBy,
String? remarks,
  })async {
final queryParams=<String,String>{};
if (receivedBy!=null&& receivedBy.isNotEmpty) queryParams['receivedBy']= receivedBy;
if (remarks!=null&& remarks.isNotEmpty) queryParams['remarks']= remarks;

final uri=Uri.parse('$baseUrl/api/delivery-orders/$deliveryOrderId/confirm')
        .replace(queryParameters: queryParams.isNotEmpty? queryParams:null);
```

```
final response=await http.put(uri, headers: {'Authorization':'Bearer$token'});

if (response.statusCode==200)returnjsonDecode(response.body)asMap<String,dynamic>;
final error=jsonDecode(response.body);
throwException(error['message']??'Failed to confirm delivery (${response.statusCode})');
  }

// ✅ Confirm Partial Delivery — selected garments only (matches web modal checkboxes)
Future<Map<String,dynamic>>confirmPartialDelivery({
requiredint deliveryOrderId,
requiredList<int> garmentIds,// garmentId values of checked items
String? proofImageUrl,// optional photo URL
String? receivedBy,
String? remarks,
  })async {
final queryParams=<String,String>{};
if (receivedBy!=null&& receivedBy.isNotEmpty) queryParams['receivedBy']= receivedBy;
if (remarks!=null&& remarks.isNotEmpty) queryParams['remarks']= remarks;

final uri=Uri.parse('$baseUrl/api/delivery-orders/$deliveryOrderId/confirm-partial')
        .replace(queryParameters: queryParams.isNotEmpty? queryParams:null);

final body=jsonEncode({
'garmentIds': garmentIds,
if (proofImageUrl!=null&& proofImageUrl.isNotEmpty)'proofImageUrl': proofImageUrl,
    });

final response=await http.put(
      uri,
      headers: {'Authorization':'Bearer$token','Content-Type':'application/json'},
      body: body,
    );

if (response.statusCode==200)returnjsonDecode(response.body)asMap<String,dynamic>;
final error=jsonDecode(response.body);
throwException(error['message']??'Failed to confirm partial delivery (${response.statusCode})');
  }
}
```

Step 2 — Create Delivery Model

```
dart
// lib/models/delivery_order.dart

classDeliveryOrder {
finalint id;
finalString deliveryOrderNumber;
finalString customerName;
finalString? branchName;
finalString driverName;
finalString status;
finalString? dispatchTime;
finalString? deliveryTime;
finalString? receivedBy;
finalString? remarks;
finalint totalItems;
finalList<DeliveryItem> items;
```

```
DeliveryOrder({
requiredthis.id,
requiredthis.deliveryOrderNumber,
requiredthis.customerName,
this.branchName,
requiredthis.driverName,
requiredthis.status,
this.dispatchTime,
this.deliveryTime,
this.receivedBy,
this.remarks,
requiredthis.totalItems,
requiredthis.items,
  });

factoryDeliveryOrder.fromJson(Map<String,dynamic> json) {
returnDeliveryOrder(
      id: json['id'],
      deliveryOrderNumber: json['deliveryOrderNumber'],
      customerName: json['customerName'],
      branchName: json['branchName'],
      driverName: json['driverName'],
      status: json['status'],
      dispatchTime: json['dispatchTime'],
      deliveryTime: json['deliveryTime'],
      receivedBy: json['receivedBy'],
      remarks: json['remarks'],
      totalItems: json['totalItems'],
      items: (json['items']asList<dynamic>)
          .map((item)=>DeliveryItem.fromJson(item))
          .toList(),
    );
  }

boolget isDeliverable=>
      status=='DISPATCHED'|| status=='IN_TRANSIT'|| status=='PARTIALLY_DELIVERED';

boolget isDelivered=> status=='DELIVERED';

boolget isPartiallyDelivered=> status=='PARTIALLY_DELIVERED';
}

classDeliveryItem {
finalint id;
finalint garmentId;
finalString garmentQrCode;
finalString productName;
finalString status;

DeliveryItem({
requiredthis.id,
requiredthis.garmentId,
requiredthis.garmentQrCode,
requiredthis.productName,
requiredthis.status,
  });

factoryDeliveryItem.fromJson(Map<String,dynamic> json) {
returnDeliveryItem(
      id: json['id'],
      garmentId: json['garmentId'],
```

```
      garmentQrCode: json['garmentQrCode'],
      productName: json['productName']??'Unknown',
      status: json['status'],
    );
  }
}
```

Step 3 — Confirm Delivery Screen (UI Example)

```
dart
// lib/screens/confirm_delivery_screen.dart

import'package:flutter/material.dart';

classConfirmDeliveryScreenextendsStatefulWidget {
finalDeliveryOrder delivery;
finalDeliveryApiService apiService;

constConfirmDeliveryScreen({
Key? key,
requiredthis.delivery,
requiredthis.apiService,
  }):super(key: key);

@override
State<ConfirmDeliveryScreen>createState()=>_ConfirmDeliveryScreenState();
}

class_ConfirmDeliveryScreenStateextendsState<ConfirmDeliveryScreen> {
finalTextEditingController _receivedByController=TextEditingController();
finalTextEditingController _remarksController=TextEditingController();
bool _isLoading=false;
String? _errorMessage;

Future<void>_confirmDelivery()async {
setState(() {
      _isLoading=true;
      _errorMessage=null;
    });

try {
final result=await widget.apiService.confirmDelivery(
        deliveryOrderId: widget.delivery.id,
        receivedBy: _receivedByController.text.trim(),
        remarks: _remarksController.text.trim(),
      );

// Success!
if (mounted) {
ScaffoldMessenger.of(context).showSnackBar(
constSnackBar(
            content:Text('✅ Delivery confirmed successfully!'),
            backgroundColor:Colors.green,
          ),
        );
Navigator.pop(context, result);// Return updated data
      }
    }catch (e) {
```

```
setState(() {
        _errorMessage= e.toString().replaceFirst('Exception: ','');
      });
    }finally {
setState(() {
        _isLoading=false;
      });
    }
  }

@override
Widgetbuild(BuildContext context) {
returnScaffold(
      appBar:AppBar(
        title:Text('Confirm Delivery #${widget.delivery.id}'),
        backgroundColor:constColor(0xFF1A237E),
      ),
      body:SingleChildScrollView(
        padding:constEdgeInsets.all(16),
        child:Column(
          crossAxisAlignment:CrossAxisAlignment.start,
          children: [
// Order Info Card
Card(
              child:ListTile(
                title:Text(widget.delivery.deliveryOrderNumber),
                subtitle:Text(widget.delivery.customerName),
                trailing:Chip(
                  label:Text(widget.delivery.status),
                  backgroundColor:Colors.blue.shade100,
                ),
              ),
            ),
constSizedBox(height:16),

// Items Summary
Text(
'${widget.delivery.totalItems} items to deliver',
              style:constTextStyle(fontWeight:FontWeight.bold, fontSize:16),
            ),
constSizedBox(height:8),
            ...widget.delivery.items.map((item)=>ListTile(
              leading:constIcon(Icons.checkroom),
              title:Text(item.productName),
              subtitle:Text(item.garmentQrCode),
              trailing:Chip(
                label:Text(item.status),
                backgroundColor:Colors.orange.shade100,
              ),
            )),

constSizedBox(height:24),
constDivider(),
constText(
'Delivery Confirmation',
              style:TextStyle(fontWeight:FontWeight.bold, fontSize:16),
            ),
constSizedBox(height:12),

// Received By Field
TextField(
              controller: _receivedByController,
```

```
              decoration:constInputDecoration(
                labelText:'Received By (Customer Name)',
                hintText:'Enter name of person who received items',
                border:OutlineInputBorder(),
                prefixIcon:Icon(Icons.person),
              ),
            ),
constSizedBox(height:12),

// Remarks Field
TextField(
              controller: _remarksController,
              maxLines:3,
              decoration:constInputDecoration(
                labelText:'Remarks (Optional)',
                hintText:'Any notes about the delivery...',
                border:OutlineInputBorder(),
                prefixIcon:Icon(Icons.notes),
              ),
            ),
constSizedBox(height:16),

// Error message
if (_errorMessage!=null)
Container(
                padding:constEdgeInsets.all(12),
                color:Colors.red.shade50,
                child:Text(
'⚠$_errorMessage',
                  style:constTextStyle(color:Colors.red),
                ),
              ),

constSizedBox(height:24),

// Confirm Button
SizedBox(
              width:double.infinity,
              height:52,
              child:ElevatedButton(
                onPressed: widget.delivery.isDeliverable&&!_isLoading
? _confirmDelivery
:null,
                style:ElevatedButton.styleFrom(
                  backgroundColor:constColor(0xFF1A237E),
                  shape:RoundedRectangleBorder(
                    borderRadius:BorderRadius.circular(8),
                  ),
                ),
                child: _isLoading
?constCircularProgressIndicator(color:Colors.white)
:constText(
'Confirm Delivery',
                        style:TextStyle(fontSize:16, color:Colors.white),
                      ),
              ),
            ),

// Show if already delivered
if (widget.delivery.isDelivered)
constCenter(
                child:Padding(
```

```
                  padding:EdgeInsets.all(16),
                  child:Text(
'✅ This delivery has already been confirmed.',
                    style:TextStyle(color:Colors.green, fontSize:15),
                  ),
                ),
              ),
          ],
        ),
      ),
    );
  }
}
```

7. Full Mobile Screen Flow

```
📱 DRIVER APP FLOW
═══════════════════════════════════════════

1. Login Screen
   → POST /api/auth/login
   → Save JWT token

2. My Deliveries List Screen
   → GET /api/delivery-orders/my-deliveries
   → Show list of assigned delivery orders

3. Delivery Detail Screen (tap one)
   → GET /api/delivery-orders/{id}
   → Show items, customer info, status

4. Confirm Delivery Screen
   → Show garment list with checkboxes
   → Enter "Received By" name
   → Optional photo (proof of delivery)
   → Tap "Confirm Delivery"
   → PUT /api/delivery-orders/{id}/confirm?receivedBy=...&remarks=...

5. Success Screen
   → Show "Delivered!" confirmation
   → Status changes to DELIVERED
   → Go back to My Deliveries list
```

8. Quick Reference — cURL Examples

Login

```
bash
curl-XPOSThttp://localhost:8080/api/auth/login\
-H"Content-Type: application/json"\
-d'{"email":"driver@star.com","password":"password123"}'
```

Get My Deliveries

```
bash
curl-XGEThttp://localhost:8080/api/delivery-orders/my-deliveries\
-H"Authorization: Bearer YOUR_TOKEN_HERE"
```

Get Single Delivery

```
bash
curl-XGEThttp://localhost:8080/api/delivery-orders/1284\
-H"Authorization: Bearer YOUR_TOKEN_HERE"
```

Confirm Delivery

```
bash
curl-XPUT"http://localhost:8080/api/delivery-orders/1284/confirm?receivedBy=Mohammed&remarks=Deli
vered+at+door"\
-H"Authorization: Bearer YOUR_TOKEN_HERE"
```

Download PDF Receipt

```
bash
curl-XGEThttp://localhost:8080/api/delivery-orders/1284/pdf\
-H"Authorization: Bearer YOUR_TOKEN_HERE"\
-odelivery_1284.pdf
```

---

9. Summary Table — All Delivery APIs

#	Method	Endpoint	Auth	Purpose
1	`POST`	`/api/auth/login`	None	Driver login, get JWT token
2	`GET`	`/api/delivery-orders/my-deliveries`	DRIVER	Get all deliveries assigned to me
3	`GET`	`/api/delivery-orders/{id}`	Any	Get single delivery order detail
4	`PUT`	`/api/delivery-orders/{id}/confirm`	DRIVER / ADMIN	⭐ **Confirm delivery**
5	`GET`	`/api/delivery-orders/driver/{driverId}`	Any	Get deliveries by driver ID

#	Method	Endpoint	Auth	Purpose
6	`GET`	`/api/delivery-orders/status/{status}`	Any	Filter by status
7	`GET`	`/api/delivery-orders/{id}/pdf`	Any	Download PDF receip